

Homework Exercises 5

Dr. Sarvar Abdullaev
s.abdullaev@inha.uz

October 16, 2022

After completing all lab exercises below, zip all your solution files into a single archive with your student ID (e.g. u210023.zip) and upload it to eClass before the deadline.

Note, you do not have to submit other lab materials such as lab slides or examples.

1 Command Line Interface

The command-line interface, sometimes referred to as the CLI, is a tool into which you can type text commands to perform specific tasks—in contrast to the mouse’s pointing and clicking on menus and buttons. Since you can directly control the computer by typing, many tasks can be performed more quickly, and some tasks can be automated with special commands that loop through and perform the same action on many files—saving you, potentially, loads of time in the process.

1.1 Practice Bash in Linux

1.1.1 Setting up Linux environment in CoCalc

You don’t need to install Linux on your computer in order to interact with Bash shell for Linux. Instead you can sign up to CoCalc website and gain access to a real Linux terminal from your web browser. Follow the instructions given Figures 1,2,3,4,5 and 6.

1.1.2 Bash Basics

To get started with the command line, you’ll need to open up a terminal window and get ready to start typing commands. Here’s a list of basic commands you can use, organized by the type of activity that you might want to perform.

When you run your terminal application (WLS2 on Windows, Terminal on Mac and Linux), your command prompt will start up pointing to a specific folder on your hard drive. You can navigate between folders, act on files inside those folders, or perform other actions.

- List all files in your current directory. The `ls` command does not show hidden files by default. Hidden files have file names that begin with a dot (.). To list all files in your home directory including hidden files, use the `-a` option with the `ls` command.

List all contents of your current working directory, including hidden files:

```
ls -a
```

- Make directory: `mkdir`. To make a directory called `intro2it` inside your current working directory, type

```
mkdir intro2it
```

To see the directory you have just created, type

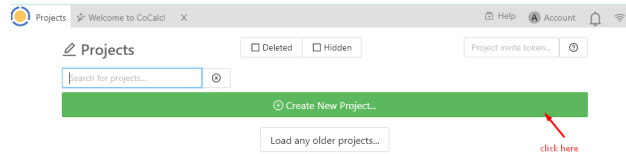


Figure 1: Creating New Project in CoCalc

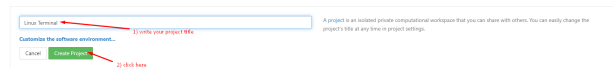


Figure 2: Naming New Project in CoCalc

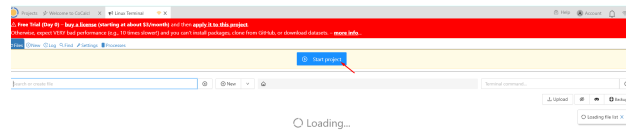


Figure 3: Start your new project

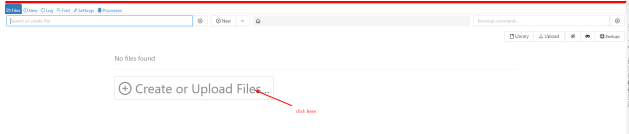


Figure 4: Create a new file in your project

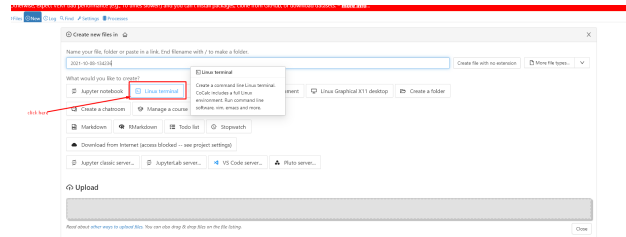


Figure 5: Create a Linux Terminal file

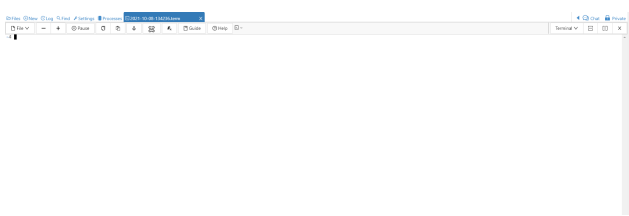


Figure 6: Write all your learned commands in live Linux terminal

```
ls
```

- Change to a different directory: **cd**. The command **cd directory** means change the current working directory to 'directory'. The current working directory may be thought of as the directory you are in, i.e. your current position in the filesystem. To change to the directory you have just made, type

```
cd intro2it
```

To go up one level in the directory tree, use two dots:

```
cd ..
```

At this point you should be back in your home directory.

- To jump back to your home directory (regardless of how deep in the file tree you happen to be), use either of the two following commands:

```
cd ~
```

or

```
cd
```

- Show the current directory: **pwd**. Pathnames enable you to work out where you are in relation to the whole file system. For example, to find out the absolute pathname of your current directory, use:

```
pwd
```

The full pathname will look something like **/home/jshafer** which means that **jshafer** (your home directory) is inside the directory **home** (the directory containing all user accounts).

- Copy file: **cp**. Before doing these commands, enter the **intro2it** directory, and create two empty files using the **touch** command:

```
cd intro2it
touch file1
```

To make a copy of **file1** and give it the name **file2** (in the current working directory), use:

```
cp file1 file2
```

Now you should have two files: **file1** and **file2**.

Note that you can use relative or absolute paths to identify files. For example, to copy **file1** from the current working directory and save it as the name **file2** in one directory up from the current directory, use:

```
cp file1 ../file2
```

- Move file: **mv**. To move (or rename) **file1** into **file2**, use:

```
mv file1 file2
```

- Delete file: **rm**. To delete ("remove") a file, use:

```
rm file2
```

Caution! The **rm** command is very powerful. Once you remove a file, recovering that file can be very difficult, if not impossible. The command line does not provide the safety of "are you sure?" questions like the GUI interface does. Further, no "unremove" or "undelete" command. If you're paranoid about deleting something accidentally, use this instead:

```
rm -i file2
```

- Delete ("remove") directory: **rmdir**.

```
rmdir directoryname
```

Note that the directory must be empty first! If it is not empty, and you want to remove everything in the directory along with the directory itself, use:

```
rm -rf directoryname
```

- Clear the terminal window: **clear**

```
clear
```

- View a (text) file on screen: **less**

```
less /var/log/syslog
```

The command **less** writes the contents of **syslog** (located in the **/var/log/** directory) onto the screen a page at a time.

- Press the [space-bar] if you want to see another page
- Press the [arrow keys up/down] if you want to scroll
- Press [q] if you want to quit reading.

- Read documentation on a command: **man**. For example, if you want learn about attributes or the functionality of some unknown command use, **man commandname**. For example,

```
man nano
```

If you cannot remember the command name, but know some keywords, you search this command using following command:

```
man -k keyword
```

For example if you search for:

```
man -k editor
```

You will see all available terminal editors on your console.

- Create and edit text files in terminal using **nano** program:

```
nano sample.txt
```

- Press **Ctrl+X** to exit Nano
- Press **Ctrl+O** to save text being edited in Nano
- Use arrows to move the cursor
- Use **Ctrl+K** to cut and **Ctrl+U** to paste selected text in Nano
- Use **Alt+U** to undo and **Alt+E** to redo your last action in Nano

If accidentally exit from Nano by pressing **Ctrl+Z**, you can return back to nano by issuing following command:

```
fg
```

If you have an access to a real Linux/macOS machine, you can try all CLI commands you have learned above on this machine too. You can find more detailed and comprehensive tutorial for all Linux CLI commands and Bash scripting language in this tutorial. Practice these commands in your Linux terminal.

1.2 Practice Batch in Windows (optional)

If you want to do similar operations using Windows's CLI shell - Batch Script, you can watch video tutorials given here. You can also write executable scripts using Batch as shown in this tutorial.

2 Python Tasks

Watch **Week 4: Structured Types** lecture videos on this course. Then complete following Python tasks.

1. Write a Python function to check whether a number is *perfect number* or not. In number theory, a perfect number is a positive integer that is equal to the sum of its proper positive divisors, that is, the sum of its positive divisors excluding the number itself (also known as its aliquot sum). Equivalently, a perfect number is a number that is half the sum of all of its positive divisors (including itself).

For example, the first perfect number is 6, because 1, 2, and 3 are its proper positive divisors, and $1+2+3 = 6$. Equivalently, the number 6 is equal to half the sum of all its positive divisors: $\frac{1+2+3+6}{2} = 6$. The next perfect number is $28 = 1 + 2 + 4 + 7 + 14$. This is followed by the perfect numbers 496 and 8128. Try to find all perfect numbers till 1,000,000.

Save your code as **ps2-1.py** and output as **ps2-1.txt**

2. Write a Python function that prints out the first n rows of Pascal's triangle. Each number is the two numbers above it added together

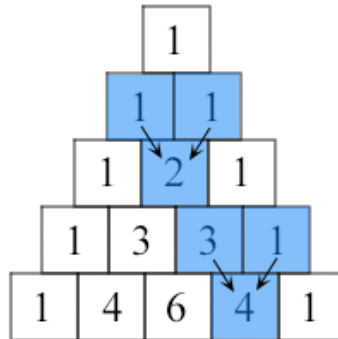


Figure 7

Save your program as `ps2-2.py`.

3. Write a Python function to check whether a string is a pangram or not. Pangrams are words or sentences containing every letter of the alphabet at least once. For example, *"The quick brown fox jumps over the lazy dog"*.

Save your program as `ps2-3.py`.

4. Write a Python function that takes a list and returns a new list with unique elements of the first list.

```
input = [1,2,3,3,3,3,4,5]
output = [1,2,3,4,5]
```

Save your program as `ps2-4.py`.

5. Write a Python program to combine two dictionary adding values for common keys.

```
d1 = {'a': 100, 'b': 200, 'c':300}
d2 = {'a': 300, 'b': 200, 'd':400}

output = {'a': 400, 'b': 400, 'd': 400, 'c': 300}
```

Save your program as `ps2-5.py`.